Lab Report Cover Page (Please print this and attach to the cover of your lab report)

LAB NO.:	**05**
SUBJECT:	**Mobile Application Development**
NAME:	**Esha Khalid**
REGISTRATION:	**FA23-BCE-030**
SUBMITTED TO:	**Dr. Ali Raza Shahid**
DATE:	**17-10-2026**

Lab Assessment (To be filled by the lab-instructor)

In Lab Evaluation			/ 7	/10
Lab Report	Data Analysis		/ 1	
	Data Presentation		/ 1	
	Writing Style		/ 1	

Comments:

Instructor Name: **Dr. Ali Raza** Instructor Signature

Report (To be submitted by the student)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. Summary of skills learned
3. Conclusion

Note: Keep your report concise and to-the-point.

LAB #05

JETPACK COMPOSE UI STATE VALIDATION AND ACCESSIBILITY

Course / alignment	Laboratory details

CSC303 - Mobile Application Development	Kotlin and Jetpack Compose Android application
CLO2; PLO5 - Modern Tool Usage	State, validation, resources and accessibility
Assessment: 10 marks	In-lab demonstration: 7; report: 3

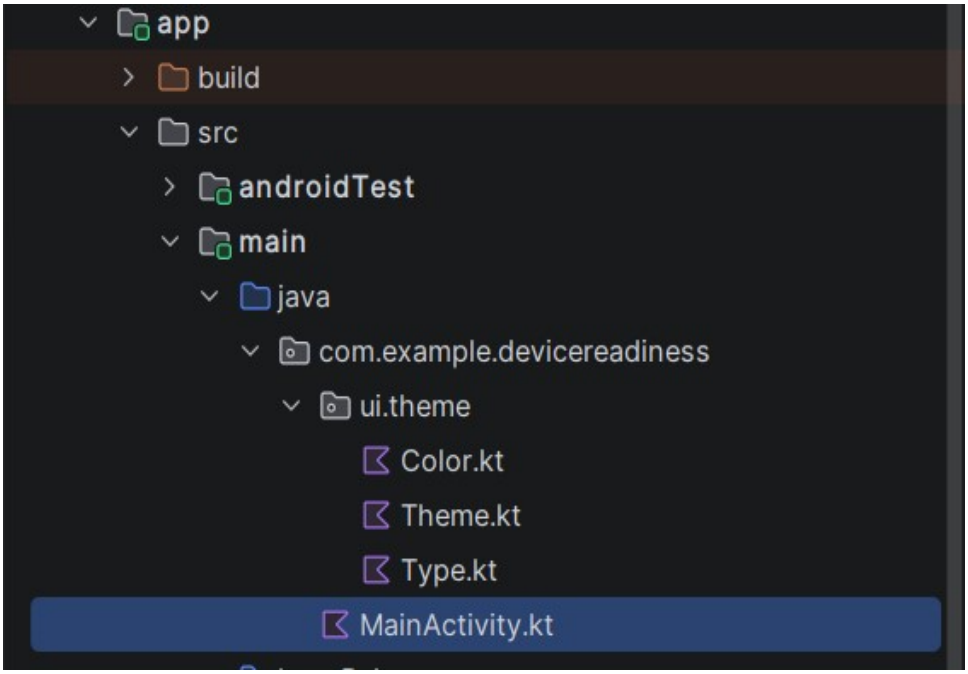
1. Objectives

Build a single-screen Device Readiness Checker using Kotlin, Jetpack Compose and Material 3. The application accepts device information, validates user input, derives a readiness result, preserves form state during activity recreation, uses Android resources for user-facing text and exposes clear semantics for accessibility services and UI tests.

5. Guided Laboratory Procedure

Task 1 - Create the Compose project

1. Create an Android Studio project named DeviceReadiness using the Empty Activity template and Kotlin. Store it in a lab05-device-readiness folder inside the laboratory repository.
2. Retain the Compose and Material 3 configuration generated by the template. Build and run the starter application before changing the UI.
3. Locate MainActivity.kt, the project theme package and res/values/strings.xml. Identify which code runs inside setContent.
4. Replace only the generated sample composables. Keep the project theme and enableEdgeToEdge setup unless the laboratory configuration requires otherwise.



Laboratory configuration

```
> Task :app:createDebugApkListingFileRedirect

BUILD SUCCESSFUL in 1m 7s
36 actionable tasks: 36 executed
Configuration cache entry stored.
```

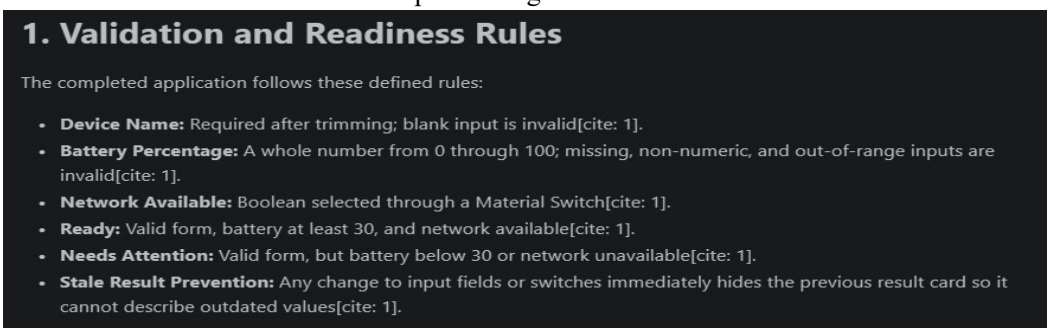
Build succesful

Task 2 - Define the validation and readiness rules

The completed form follows these rules:

Field or decision	Rule
Device name	Required after trimming; blank input is invalid.
Battery percentage	A whole number from 0 through 100; missing, non-numeric and out-of-range input is invalid.
Network available	Boolean selected through a Material Switch.
Ready	Valid form, battery at least 30 and network available.
Needs attention	Valid form but battery below 30 or network unavailable.
After input changes	Hide the previous result so it cannot describe stale values.

Record the rule table in the README before implementing it. The UI and tests must use the same boundaries.



README Validation table

Task 3 - Add strings and formatted resources

Replace hard-coded interface text with entries in res/values/strings.xml. Add at least the following resources.

- Use stringResource for ordinary and formatted text in composable functions.
- Do not construct accessibility descriptions by concatenating untranslated English fragments.
- Keep field labels visible; a content description is not a replacement for a visible form label.

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Device Readiness</string>
    <string name="screen_title">Device Readiness Checker</string>
    <string name="screen_intro">Enter the device information and evaluate deployment readiness.</string>
    <string name="device_name_label">Device name</string>
    <string name="battery_label">Battery percentage</string>
    <string name="battery_support">Enter a whole number from 0 to 100</string>
    <string name="network_label">Network available</string>
    <string name="evaluate">Evaluate</string>
    <string name="clear">Clear</string>
    <string name="error_device_name_required">Device name is required</string>
    <string name="error_battery_invalid">Enter a valid battery percentage from 0 to 100</string>
    <string name="result_ready">%1$s is ready for deployment.</string>
    <string name="result_attention">%1$s requires attention before deployment.</string>
    <string name="status_ready">Ready</string>
    <string name="status_attention">Needs attention</string>
    <string name="network_available">available</string>
    <string name="network_unavailable">unavailable</string>
    <string name="network_switch_description">Change network availability</string>
    <string name="battery_progress_description">Battery level %1$d percent</string>
    <string name="result_accessibility">Device %1$s. Battery %2$d percent. Network %3$s. Status %4$s.</s
</resources>
```

string.xml

Task 4 - Create the stateful route and hoist screen state

DeviceReadinessApp owns small UI element state. The child screen receives current values and event callbacks, which makes the screen easier to preview and test.

```
DeviceReadinessScreen(
    deviceName = deviceName,
    batteryText = batteryText,
    networkAvailable = networkAvailable,
    nameError = nameError,
    batteryError = batteryError,
    resultReady = resultReady,
    onDeviceNameChange = { deviceName = it; submitted = false },
    onBatteryChange = { batteryText = it; submitted = false },
    onNetworkChange = { networkAvailable = it; submitted = false },
    onEvaluate = { submitted = true },
    onClear = {
```

Adding DeviceReadingApp and DeviceReadingScreen

Task 5 - Build the stateless Material 3 screen

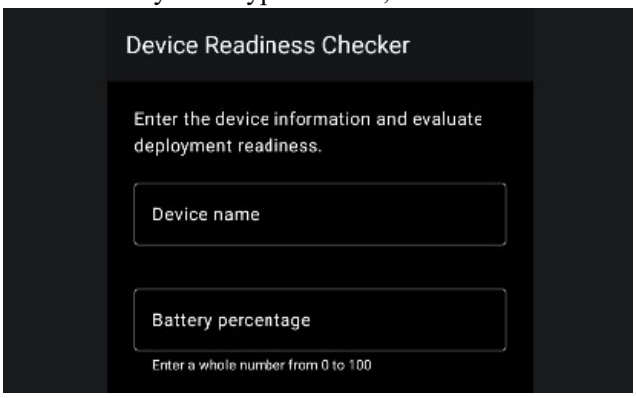
Add the screen signature first, then implement its content. Do not create new remember state inside this function for the four hoisted values.



Build succesful task5 Device Readiness Checker

Task 6 - Add validated text fields

Place these fields inside the Column. The keyboard type is a hint; validation must still reject non-numeric input.

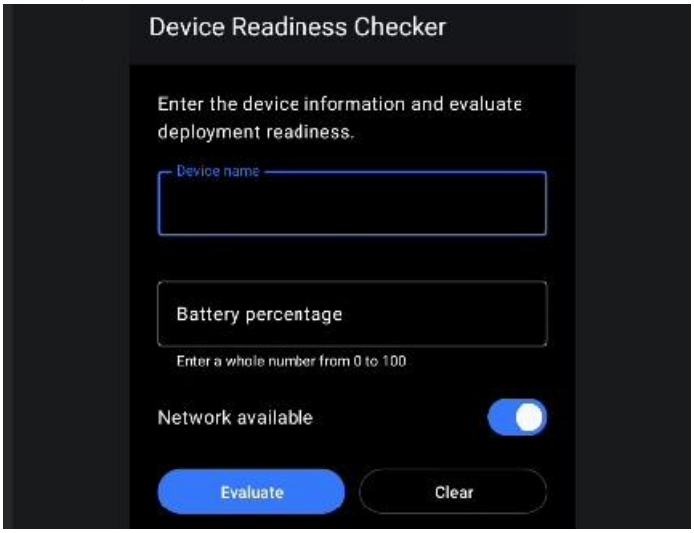


Added validation text fields

Task 7 - Add the switch and actions

Create a labelled switch row and two Material actions. The switch description comes from a resource.

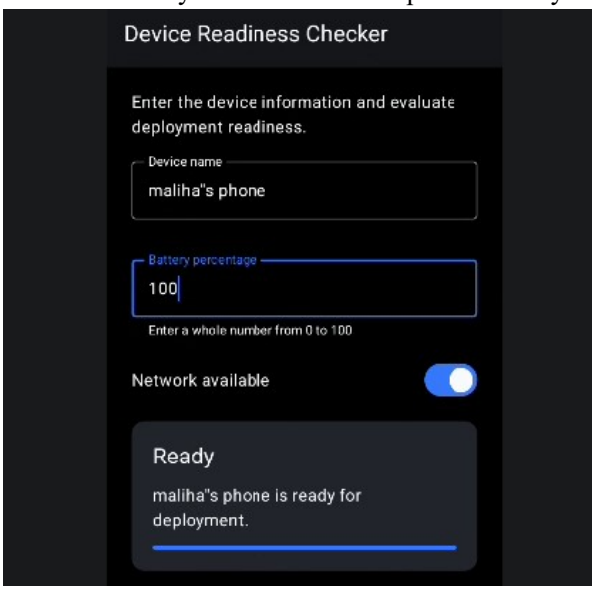
- Use standard Material controls so focus, role and interaction semantics are available by default.
- Keep controls large enough for reliable touch interaction and verify the layout with increased font size. Do not use colour as the only indication of readiness or validation failure.



Switches and actions

Task 8 - Display an accessible readiness result

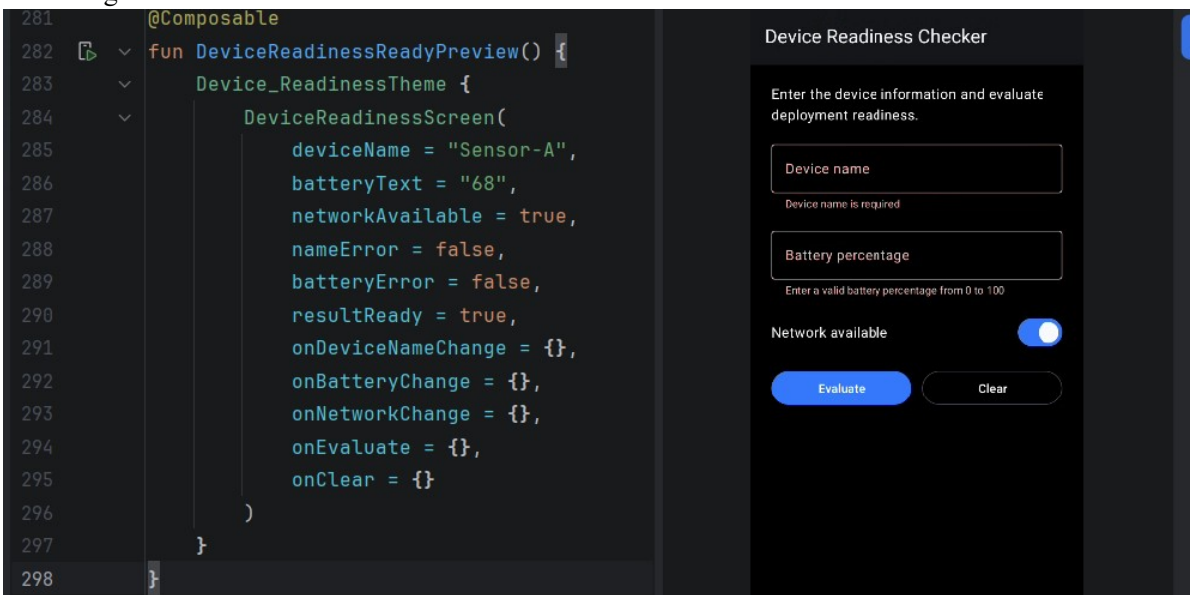
Call ReadinessResultCard only when resultReady is not null and the parsed battery value is valid.



Readiness result

Task 9 - Add previews

Create previews for an empty form and a completed readiness result. Preview callbacks use empty lambdas because they do not change state.



Adding preview to MainAvtivity.kt

Task 10 - Verify state, validation and accessibility

Execute every row and record the actual result. Restore valid data after each invalid case.

ID	Input or action	Expected behaviour
T1	Clean build and launch	Material 3 screen opens with labelled fields, switch and actions.
T2	Evaluate blank form	Both visible validation messages appear; no result card is shown.

T3	Battery text, -1 or 101	Battery error appears and the form remains editable.
T4	Battery 0, 29, 30 and 100 with network on	Boundaries are valid; 0/29 need attention and 30/100 are ready.
T5	Battery 80; toggle network off	Result changes to needs attention only after Evaluate is selected again.
T6	Evaluate valid values, then edit a field	The old result card disappears so stale information is not displayed.
T7	Enter values and recreate the activity	Supported form values are restored through rememberSaveable.
T8	Increase font size and use accessibility focus	Content remains reachable; labels, switch and result are announced meaningfully.

Accessibility review

- Navigate using an accessibility service or the Compose semantics inspector and record the focus order.
- Confirm that each field has a visible label and that validation text is understandable without colour.
- Confirm that the switch exposes both its role/state and a meaningful description.
- Confirm that readiness and battery information can be understood when the screen is not viewed.
- Check increased font size, narrow portrait layout and landscape layout for clipped or unreachable content.

Task 11 - Add focused Compose UI tests

Create DeviceReadinessScreenTest.kt under app/src/androidTest in the application package.

- Run both instrumented tests on the selected emulator or device and retain the successful result as evidence. Use test tags for stable technical selection while keeping visible labels and semantics meaningful for users.
- If a test cannot find a node, inspect the merged and unmerged semantics trees before changing the UI blindly.



Student and environment	Details
Name and registration number	EshaKhalid, FA23-BCE-030
Android Studio / SDK / device	Project: lab5DeviceReadiness (Kotlin, Jetpack Compose, Material 3), package com.example.devicereadiness.

Critical Analysis of the Lab Activity

In this lab, I successfully built a Device Readiness Checker using Jetpack Compose and Material 3, implementing unidirectional data flow via state hoisting to separate state management from the stateless UI. The application features robust input validation that prevents crashes and reveals error messages only after evaluation, a dynamic result card that clears upon editing, and full string resource integration. Accessibility was prioritized through visible field labels, clear text-based feedback, and detailed semantic descriptions for interactive components. By resolving initial compilation errors and verifying edge cases, I achieved all core lab objectives, with future improvements planned for broader TalkBack testing across various orientations, screen sizes, and font scales.

State and Event Map

State	Displayed by	Changed by	Saved with
deviceName	Device name field and result card	onDeviceNameChange, Clear	rememberSaveable
batteryText	Battery field, error text, result card	onBatteryChange, Clear	rememberSaveable
networkAvailable	Switch and readiness decision	onNetworkChange, Clear	rememberSaveable
submitted	Error visibility and result visibility	Evaluate (true); any edit and Clear (false)	rememberSaveable

Verification Table (T1-T8)

ID	Input or action	Expected behaviour	Actual result	Pass/Fail
T1	Clean build and launch	Material 3 screen opens with labelled fields, switch and actions.	Build was successful. The app opened with Device name, Battery percentage, Network available switch, and Evaluate and Clear buttons.	Pass
T2	Evaluate blank form	Both validation messages appear; no result card.	"Device name is required" and "Enter a valid battery percentage from 0 to 100" were shown. No result card.	Pass
T3	Battery text, -1 or 101	Battery error appears; form stays editable.	Battery error was shown for text, -1 and 101. Fields could still be edited.	Pass
T4	Battery 0, 29, 30, 100 with network on	All valid; 0 and 29 need attention; 30 and 100 are ready.	0 and 29 showed Needs attention. 30 and 100 showed Ready (100 is in the screenshot).	Pass
T5	Battery 80; switch network off	Result changes only after Evaluate again.	Result did not change when the switch was toggled. After Evaluate it showed Needs attention.	Pass
T6	Evaluate valid values, then edit a field	Old result card disappears.	The card disappeared as soon as I edited a field, because submitted became false.	Pass
T7	Enter values and recreate activity	Values are restored by rememberSaveable.	Name, battery and switch values were still there after rotating the screen.	Pass
T8	Increase font size and use accessibility focus	Content reachable; labels, switch and result announced properly.	All content could be scrolled to. Labels, the switch and the result were read with meaningful text.	Pass